Action Spotting in Soccer Broadcast Videos

CS518: Deep Learning for Computer Vision
Abhinav Sandru · Gaurav Chintakunta · Raviteja Ravella
GitHub: https://github.com/AbhinavSandru/cs518-action-spotting

1. Introduction

Broadcast soccer matches are rich sources of tactical and statistical information, but extracting structured event data from raw video requires significant manual effort. **Action spotting** is the task of automatically identifying the precise timestamps of predefined actions in untrimmed long-form video. Unlike action recognition, which classifies a pre-segmented clip, action spotting must locate when events occur within a full 90-minute match without any prior segmentation.

This project addresses action spotting on the SoccerNet-v2 benchmark [1], which defines 17 action classes (goals, fouls, cards, corners, shots, substitutions, etc.) across 500 professional broadcast soccer games. The core challenge is **extreme temporal sparsity**: actions occur at fewer than one frame per thousand per class, making naive classification approaches degenerate — a model can achieve near-zero loss by predicting nothing.

We propose a lightweight **Transformer encoder** trained on precomputed ResNet-152 [2] appearance features, combined with focal loss [3] and inverse-frequency class weighting to handle the severe class imbalance. The full pipeline — preprocessing, training, inference, and evaluation — runs on a single Apple M4 Pro laptop using Metal Performance Shaders (MPS) acceleration.

Final results on the SoccerNet-v2 test set: 38.18% mAP@1s · 52.86% mAP@5s.

2. Related Work / Background / Existing Approaches

2.1 Action Recognition vs. Action Spotting

Standard action recognition (e.g., on Kinetics [4]) classifies pre-trimmed clips. Action spotting is strictly harder: the model must both detect and localize events in untrimmed video, producing (timestamp, class) pairs rather than a single label.

2.2 SoccerNet Benchmark

SoccerNet [1] introduced large-scale soccer action spotting. SoccerNet-v2 [5] extended the dataset to 500 games and 17 classes with tighter annotation quality. Evaluation uses **Average-mAP** — the mean of per-class average precision scores computed at multiple temporal tolerances (1s, 2s, 5s).

2.3 Existing Approaches

NetVLAD++ [6] is the official SoccerNet-v2 baseline. It uses a NetVLAD pooling layer to aggregate temporal context from sliding windows over precomputed features, achieving ~49% mAP@5s.

CALF [7] adds a calibration loss that enforces temporal ordering of predictions, reaching ~42% mAP@5s on SoccerNet-v1.

E2E-Spot [8] trains an end-to-end model directly on raw video frames using a lightweight CNN backbone, achieving 60%+ mAP on SoccerNet-v2 by learning sport-specific features rather than relying on ImageNet-pretrained representations.

Our approach differs from NetVLAD++ by using a Transformer encoder with self-attention instead of learnable pooling, and from E2E-Spot by operating on frozen precomputed features — making it feasible to train on a single laptop. We incorporate focal loss (from object detection [3]) into the action spotting domain, which significantly improves performance on rare classes.

3. Methodology

3.1 Problem Statement

Input: A full soccer broadcast match represented as precomputed ResNet-152 feature vectors $\boldsymbol{x}_t \in \mathbb{R}^{2048}$ at $t = 1, \ldots, T$ frames, where $T \approx 10,800$ per half at 2fps.

Output: A ranked list of predictions $\{(h_i, p_i, c_i, s_i)\}$ where $h_i \in \{1, 2\}$ is the match half, p_i is the timestamp in milliseconds, $c_i \in \{1, \ldots, 17\}$ is the action class, and $s_i \in [0, 1]$ is the confidence score.

Per-frame formulation: The model performs binary classification at every frame for each of 17 classes independently. Timestamps are derived post-hoc via non-maximum suppression on the per-frame score arrays.

3.2 Preprocessing

Raw feature files for each game are concatenated across both halves to form a single array of shape $(T, 2048)$. A per-frame binary label array of shape $(T, 17)$ is constructed from the JSON annotations.

To avoid penalizing the model for being one frame off on a 2fps signal, we apply **Gaussian label spreading**: each annotated frame t^* receives a soft target using a Gaussian kernel with $\sigma = 1$ over a ± 2 frame neighborhood:

$$y_{t,c} = \exp\left(-\frac{(t - t^*)^2}{2\sigma^2}\right) \quad \text{for } |t - t^*| \leq 2$$

The full game arrays are then sliced into overlapping 60-frame windows (stride 30) and saved as compressed `.npz` files. This **preprocessing step** reduces training I/O by 8× compared to loading full game files on the fly.

3.3 Model Architecture

The model, **ActionSpotter**, is a standard Transformer encoder operating on sliding windows of frame features.

Step 1 — L2 Normalization: Input features are L2-normalized along the feature dimension, projecting each vector onto the unit hypersphere. This removes scale variance across different games, stadiums, and broadcast cameras.

Step 2 — Linear Projection: A learned linear layer maps from 2048 to $d_{\text{model}} = 256$ dimensions.

Step 3 — Positional Embedding: A learned positional embedding of shape $(1, 1000, 256)$ is added, encoding the temporal position of each frame within the window.

Step 4 — Transformer Encoder: Four encoder layers, each with: - Multi-head self-attention: $n_{\text{head}} = 4$ heads - Feed-forward network: $d_{\text{ff}} = 1024$ - Dropout: $p = 0.3$ - Layer normalization (post-attention)

Step 5 — Classification Head: A linear layer maps each frame's hidden state to 17 logits.

Output: $(B, T, 17)$ logits — one score per frame per class. Total parameters: ~1.4 million.

The key advantage of self-attention over RNNs is that every frame in the 30-second window has direct access to every other frame in $O(1)$ operations, enabling the model to learn long-range temporal patterns (e.g., buildup to a goal).

3.4 Loss Function

With a negative-to-positive ratio of ~1000:1 per class, standard binary cross-entropy (BCE) produces degenerate solutions — predicting zero for all frames achieves near-zero loss while detecting nothing.

We combine three strategies:

Focal Loss [3]: For each frame-class pair with predicted probability p_t:

$$\mathcal{L}_{\text{focal}} = -(1 - p_t)^{\gamma} \log(p_t), \quad \gamma = 2$$

The factor $(1 - p_t)^2$ down-weights easy negatives (where $p_t \approx 0$ and the model is correctly confident), forcing the gradient to concentrate on uncertain, hard positive frames.

Sqrt Inverse-Frequency Class Weights: Per-class positive weight:

$$w_c = \mathrm{clip}\left(\sqrt{\frac{N_c^-}{N_c^+}},\ 1,\ 50\right)$$

where N_c^+ and N_c^- are positive and negative frame counts for class c in the training set. The square-root dampens the raw ratio (~1000) to a stable range (~8–45). The clip at 50 prevents any single class from dominating the gradient.

Combined loss:

$$\mathcal{L} = \frac{1}{BT}\sum_{b,t}(1-p_t)^2 \cdot \mathrm{BCE}(\mathrm{logit}_{b,t},\ y_{b,t},\ w_c)$$

3.5 Training Configuration

Hyperparameter	Value
Optimizer	Adam
Learning rate	1×10^{-4}
LR scheduler	ReduceLROnPlateau (factor 0.5, patience 3)
Early stopping patience	5 epochs
Batch size	32
Gradient clipping	max norm 1.0
Window size	60 frames (30 seconds)
Stride	30 frames
d_{model}	256
Attention heads	4
Encoder layers	4
Dropout	0.3
Focal loss γ	2

Training runs on Apple M4 Pro via PyTorch MPS backend. Convergence typically occurs in 15–20 epochs (~3 hours total).

3.6 Inference

At test time the full game is processed via sliding window inference. For each window position, per-frame probabilities are accumulated and averaged across overlapping windows. **Non-Maximum Suppression (NMS)** is then applied per class: the peak-scoring frame above a threshold is selected as a prediction, and all frames within ± 10 frames are suppressed before repeating.

Per-class thresholds are tuned on the validation set by sweeping $[0.05, 0.50]$ in steps of 0.05 and selecting the threshold maximizing per-class val mAP. Most classes converge to 0.05, reflecting the model's calibrated but low-magnitude output scores.

4. Experiments

4.1 Ablation Study

We trained incrementally adding each component to measure its individual contribution:

Configuration	mAP@1s	mAP@5s
Baseline: plain BCE, no weighting	~0%	~0%
+ Sqrt inverse-frequency class weights	~24%	~39%

Configuration	mAP@1s	mAP@5s
+ Focal loss ($\gamma = 2$)	~28%	~44%
+ L2 feature normalization	31.94%	~48%
+ Per-class threshold tuning	**38.18%**	**52.86%**

The baseline produces zero detections — the model learns to predict nothing. Class weights alone recover meaningful performance. Focal loss adds further improvement by redirecting gradient toward hard examples. L2 normalization is a single-line addition that removes scale variance and gives a consistent gain. Per-class threshold tuning provides the largest single jump.

4.2 Per-Class Results (Test Set)

Class	mAP@1s	mAP@5s
Substitution	0.6951	0.8630
Foul	0.6281	0.8096
Yellow card	0.5572	0.7663
Offside	0.5807	0.6409
Throw-in	0.4852	0.6684
Yellow->red card	0.4757	0.6566
Corner	0.4514	0.6606
Goal	0.4483	0.7390
Goalkeeper saves	0.4422	0.6970
Shots off target	0.4083	0.4636
Shots on target	0.3635	0.4496
Indirect free-kick	0.3262	0.5699
Clearance	0.2850	0.3802
Direct free-kick	0.2825	0.5207
Kick-off	0.0472	0.0745
Red card	0.0143	0.0271
Ball out of play	0.0000	0.0000
Average-mAP	**0.3818**	**0.5286**

High performers: Substitution (69.5%) and Foul (62.8%) are visually distinctive — substitutions involve a player in a bib at the sideline with a board held up; fouls involve players clustering and falling. These consistent visual cues are well-captured by ResNet-152 features.

Low performers: Ball out of play (0%) is visually indistinguishable from normal play at 2fps — the ball's position relative to the line cannot be determined from appearance features alone. Red card (1.4%) is extremely rare (fewer than 1 per game), and Kick-off (4.7%) looks identical to open play.

4.3 Gap Analysis: mAP@1s vs. mAP@5s

The 14-point gap between tight (38.18%) and loose (52.86%) tolerance is largely attributable to the 2fps feature extraction rate. At 2fps, the maximum achievable temporal precision is 0.5 seconds per frame — a structural limitation that affects the 1-second tolerance metric directly. Higher-fps features would close this gap significantly.

---

5. Conclusion and Future Work

We presented a lightweight Transformer encoder for temporal action spotting on SoccerNet-v2, trained entirely on precomputed ResNet-152 features on a consumer laptop. The key contributions are:

1. Demonstrating that focal loss combined with sqrt inverse-frequency class weighting resolves the degenerate training failure caused by extreme temporal sparsity.
2. Achieving 38.18% mAP@1s and 52.86% mAP@5s — competitive with NetVLAD++ — with only ~1.4M parameters and no end-to-end backbone training.
3. A complete reproducible pipeline including preprocessing, training, inference, threshold tuning, and evaluation.

Future work:

- **End-to-end backbone fine-tuning:** Fine-tuning ResNet-152 on soccer-specific data would produce domain-adapted features and likely close the gap to top methods (60%+).
- **Multi-modal features:** Adding optical flow or audio would enable detection of visually ambiguous classes like Ball out of play.
- **Longer temporal context:** Extending beyond 30-second windows using hierarchical attention or full-game BERT-style pretraining could capture match-state context (score, time, fatigue).
- **Higher frame rate:** Using 25fps features instead of 2fps would remove the structural precision bottleneck at 1-second tolerance.

References

[1] Giancola, S., et al. “SoccerNet: A Scalable Dataset for Action Spotting in Soccer Videos.” CVPR Workshops, 2018.

[2] He, K., et al. “Deep Residual Learning for Image Recognition.” CVPR, 2016.

[3] Lin, T.Y., et al. “Focal Loss for Dense Object Detection.” ICCV, 2017.

[4] Kay, W., et al. “The Kinetics Human Action Video Dataset.” arXiv:1705.06950, 2017.

[5] Deliège, A., et al. “SoccerNet-v2: A Dataset and Benchmarks for Holistic Understanding of Broadcast Soccer Videos.” CVPR Workshops, 2021.

[6] Arun, M., et al. “NetVLAD++: Efficient Aggregation of Multi-Frame-Multi-Resolution Features for Action Spotting.” CVPR Workshops, 2021.

[7] Cioppa, A., et al. “A Context-Aware Loss Function for Action Spotting in Soccer Videos.” CVPR, 2020.

[8] Hong, J., et al. “Spotting Temporally Precise, Fine-Grained Events in Video.” ECCV, 2022.

Appendix

A. Individual Contribution

My primary responsibility was the model architecture and training pipeline. I designed the ActionSpotter Transformer encoder — selecting the four-layer configuration, d_model=256 projection, multi-head self-attention with 4 heads, and learned positional embeddings — and iterated on it until convergence was reliable. I implemented the training loop end-to-end: integrating focal loss, deriving the sqrt inverse-frequency weighting formula, adding gradient clipping after observing NaN losses with raw class weights, and wiring up the ReduceLROnPlateau scheduler. Setting up the PyTorch MPS backend on the M4 Pro required explicit device placement and careful DataLoader configuration — early runs silently fell back to CPU until I added timing instrumentation to catch it. I also ran all ablation experiments, incrementally enabling each component to quantify its individual contribution to mAP.

B. Evaluation

Ablation Study

Configuration	mAP@1s	mAP@5s
Baseline: plain BCE, no weighting	~0%	~0%
+ Sqrt inverse-frequency class weights	~24%	~39%
+ Focal loss (γ=2)	~28%	~44%
+ L2 feature normalization	31.94%	~48%
+ Per-class threshold tuning	**38.18%**	**52.86%**

Each component contributes meaningfully. The baseline predicts nothing — class weights alone recover 24% mAP@1s. Focal loss pushes hard examples, L2 normalization removes scale variance, and per-class threshold tuning provides the largest single jump from ~31% to 38.18%.

Full Per-Class Results (Test Set)

Class	mAP@1s	mAP@5s
Substitution	0.6951	0.8630
Foul	0.6281	0.8096
Yellow card	0.5572	0.7663
Offside	0.5807	0.6409
Throw-in	0.4852	0.6684
Yellow→red card	0.4757	0.6566
Corner	0.4514	0.6606
Goal	0.4483	0.7390
Goalkeeper saves	0.4422	0.6970
Shots off target	0.4083	0.4636
Shots on target	0.3635	0.4496
Indirect free-kick	0.3262	0.5699
Clearance	0.2850	0.3802
Direct free-kick	0.2825	0.5207
Kick-off	0.0472	0.0745
Red card	0.0143	0.0271
Ball out of play	0.0000	0.0000
Average-mAP	**0.3818**	**0.5286**

The 14-point gap between mAP@1s and mAP@5s is largely structural: at 2fps, one frame equals 0.5 seconds, so any prediction off by a single frame misses the 1-second tolerance entirely.

C. Implementation Details

Hardware: Apple MacBook Pro M4 Pro, 24GB unified memory. Training uses PyTorch MPS backend (~3 hours to convergence).

Data storage: SoccerNet ResNet-152 features (~200GB) on external Samsung T5 SSD. Preprocessed `.npz` windows (~70GB) on internal SSD for fast DataLoader I/O.

Key commands: - Preprocessing: `python -m src.preprocess` - Training: `python main.py --mode train` - Evaluation: `python main.py --mode evaluate` - Threshold tuning: `python -m src.tune_thresholds`

Notebook: `CS518_ActionSpotting_Demo.ipynb` — full pipeline on synthetic data, no SoccerNet download required.

D. Approaches That Did Not Work

Hard BCE with raw class weights: Using the raw neg/pos ratio (~1000×) as `pos_weight` caused loss to spike to NaN within 2 epochs. Sqrt scaling brought weights to a stable 8–45 range.

Global threshold (0.5): Model output scores rarely exceed 0.35. A single 0.5 threshold produced zero detections. Per-class threshold tuning on validation set resolved this.

pin_memory=True on MPS: Triggers a PyTorch warning and slower transfers on Apple Silicon. Setting `pin_memory=False` and `num_workers=0` gave stable training.

Eager loading full game arrays: Loading full `.npy` game files during training caused 18GB+ RAM and OOM kills. Pre-sliced `.npz` windows reduced peak RAM below 4GB.